SNES Assembly Adventure 09: Project Structure and CMake

24 Mar 2022

Welcome back, Adventurer! Last time, we learned how to read input from the joypad.

This time, we'll reorganize our code and

- use a build system (generator) like CMake
- teach CMake how to build our 65816 assembly code
- restructure the directories and files in our project

This also means there's gonna be little information here that is specific to SNES programming. But I think it is very useful to know how to organize your code beyond makefiles. You'll find all code in the SNES Assembly Adventure repository on Github.

You should at least be familiar with the basics of CMake. There's a good tutorial on the CMake site. Read the first three parts of it, and you should understand enough to be able to follow the instructions here.



Without further ado, let's get started!

A Simple Directory Structure

Let's start by creating a simple directory structure. We'll start with the joypad example from last time. Create a new directory and copy the files into it:

```
snes_cradle/
    JoypadSprite.s
    MemoryMap.cfg
    SpriteColors.pal
    Sprites.vra
```

Super easy. Now we need to figure out the goal. We want to turn these files into a ROM using the following commands:

```
$ ca65 --cpu 65816 -s -o JoypadSprite.o JoypadSprite.s
$ ld65 -C MemoryMap.cfg JoypadSprite.o -o JoypadSprite.smc
```

To do so, we need to add a new language to CMake as it does not know yet how to handle the 65816 assembly code. CMake in essence needs to know two things: how to turn a source file into an object file (first command above), and how to turn several object files into a final (executable) file (second command above).

All CMake projects start with a `CMakeLists.txt` file at the top level, so let's just add that one first.

```
# snes_cradle/CMakeLists.txt

cmake_minimum_required( VERSION 3.21 FATAL_ERROR )

project( JoypadSpriteDemo )
```

This file does not a lot. It just makes sure that a version greater than or equal to 3.21 of CMake is installed and found, and that we're creating a project called `JoypadSpriteDemo`. Place it in the project's root directory:

```
snes_cradle/
    CMakeLists.txt
    JoypadSprite.s
    MemoryMap.cfg
    SpriteColors.pal
    Sprites.vra
```

Let's test this by invoking CMake to generate the build files and running a build. Open a command line, navigate to your `snes_cradle` directory, and run:

```
$ cmake -S . -B build
-- Building for: Ninja
-- The C compiler identification is Clang 14.0.0
-- The CXX compiler identification is Clang 14.0.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: C:/msys64/clang64/bin/cc.exe - skipped
-- Detecting C compile features
```

```
-- Detecting C compile features - done
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: C:/msys64/clang64/bin/c++.exe - skip
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Configuring done
-- Generating done
-- Build files have been written to: C:/Users/clc_x/code/retrodev/snesd

$ cmake --build build
ninja: no work to do.
```

We first use `cmake -S . -B build` to create an out-of-source build in the subdirectory `build` with the project's root set to the current location (`-S .`). Then we actually run the build with `cmake --build build`. Which does nothing - for now.

This is actually what we want. Since we have not told CMake which language we are using, it defaults to C/C++ and looks for a compiler. On my system, that's clang. When we actually run the build, nothing happens because our `CMakeLists.txt` file doesn't contain any commands yet.

Change `snes_cradle/CMakeLists.txt`:

```
# snes_cradle/CMakeLists.txt

cmake_minimum_required( VERSION 3.21 FATAL_ERROR )

project( JoypadSpriteDemo
    LANGUAGES CA65816 )
```

Now re-run CMake to reconfigure your project:

```
$ cmake -S . -B build
CMake Error: Could not find cmake module file: CMakeDetermineCA65816Com
CMake Error: Error required internal CMake variable not set, cmake may
```

```
Missing variable is:
CMAKE_CA65816_COMPILER_ENV_VAR
CMake Error: Error required internal CMake variable not set, cmake may
Missing variable is:
CMAKE_CA65816_COMPILER
CMake Error: Could not find cmake module file: C:/Users/clc_x/code/retr
CMake Error at CMakeLists.txt:3 (project):
  No CMAKE_CA65816_COMPILER could be found.

  Tell CMake where to find the compiler by setting the CMake cache entr
  CMAKE_CA65816_COMPILER to the full path to the compiler, or to the co
  name if it is in the PATH.


CMake Error: Could not find cmake module file: CMakeCA65816Information.
CMake Error: CMAKE_CA65816_COMPILER not set, after EnableLanguage
-- Configuring incomplete, errors occurred!
See also "C:/Users/clc_x/code/retrodev/snesdev/snesaa/09_modular_progra
```

Calamity! CMake doesn't know the language `CA65816` - which is the name I chose for the purposes of this tutorial. Let's teach CMake our new language.

Adding a Language to CMake

If you read the error message above carefully, you'll notice that CMake is looking for several variables and files that it can't find. Like many programs, CMake searches a set of predefined locations/directories for CMake module files. One such directory is a subdirectory named `cmake` of a project's root directory.

Create a new subdirectory `cmake` and add a file named `CMakeDetermineCA65816Compiler.cmake` to it:

```
snes_cradle/
    cmake/
        CMakeDetermineCA65816Compiler.cmake
    CMakeLists.txt
```

```
    JoypadSprite.s
    MemoryMap.cfg
    SpriteColors.pal
    Sprites.vra
```

The first step of adding a language to CMake is to help CMake find a compiler for said language:

```
# snes_cradle/cmake/CMakeDetermineCA65816Compiler.cmake

# Find the ca65 assembler
find_program(
    CMAKE_CA65816_COMPILER
        NAMES "ca65"
        HINTS "${CMAKE_SOURCE_DIR}"
        DOC "ca65 assembler"
)

mark_as_advanced( CMAKE_CA65816_COMPILER )

set( CMAKE_CA65816_SOURCE_FILE_EXTENSIONS s;asm )
set( CMAKE_CA65816_OUTPUT_EXTENSION .o )
set( CMAKE_CA65816_COMPILER_ENV_VAR "FOO" )

# Configure variables set in this file for fast reload later on
configure_file( ${CMAKE_CURRENT_LIST_DIR}/CMakeCA65816Compiler.cmake.in
                ${CMAKE_PLATFORM_INFO_DIR}/CMakeCA65816Compiler.cmake )
```

CMake expects to find a variable called `CMAKE_CA65816_COMPILER` and a file called `CMakeCA65816.cmake`, so we use a configuration file in the last two lines to achieve that. Here's the content of `snes_cradle/cmake/CMakeCA65816Compiler.cmake.in`:

```
# snes_cradle/cmake/CMakeCA65816Compiler.cmake.in

set( CMAKE_CA65816_COMPILER "@CMAKE_CA65816_COMPILER@" )
set( CMAKE_CA65816_COMPILER_LOADED 1 )
```

```
set( CMAKE_CA65816_SOURCE_FILE_EXTENSIONS @CMAKE_CA65816_SOURCE_FILE_EX
set( CMAKE_CA65816_OUTPUT_EXTENSION @CMAKE_CA65816_OUTPUT_EXTENSION@ )
set( CMAKE_CA65816_COMPILER_ENV_VAR "@CMAKE_CA65816_COMPILER_ENV_VAR@"
```

Next, we need to tell CMake how to use `ca65` / `CMAKE_CA65816_COMPILER`. So we add a third file called `snes_cradle/cmake/CMakeCA65816Information.cmake`:

```
# snes_cradle/cmake/CMakeCA65816Information.cmake

# How to build objects
set( CMAKE_CA65816_COMPILE_OBJECT
    "<CMAKE_CA65816_COMPILER> --cpu 65816 \
                              -s \
                              -o <OBJECT> \
                              <SOURCE>"
)

# How to build executables
set( CMAKE_CA65816_LINK_EXECUTABLE
    "ld65 -C ${CMAKE_SOURCE_DIR}/MemoryMap.cfg \
          <OBJECTS> \
          -o <TARGET>"
)

set( CMAKE_CA65816_INFORMATION_LOADED 1 )
```

Note that the name of the memory config file is hard-coded for the sake of simplicity. We'll fix that in a moment. For now, let's add the final file CMake requires. `CMakeTestCA65816Compiler.cmake` tells CMake how to run stuff like unit tests, etc. with a given compiler. Since we don't need this, we'll simply do nothing here:

```
# snes_cradle/cmake/CMakeTestCA65816Compiler.cmake

# For now just do nothing in here
set( CMAKE_CA65816_COMPILER_WORKS 1 CACHE INTERNAL "" )
```

Now your directory structure should look like this:

```
snes_cradle/
    cmake/
        CMakeCA65816Compiler.cmake.in
        CMakeCA65816Information.cmake
        CMakeDetermineCA65816Compiler.cmake
        CMakeTestCA65816Compiler.cmake
    CMakeLists.txt
    JoypadSprite.s
    MemoryMap.cfg
    SpriteColors.pal
    Sprites.vra
```

Testing Our New Language

Now we will build our demo ROM with CMake. Let's update the `CMakeLists.txt` in our project's root directory:

```
# snes_cradle/CMakeLists.txt

cmake_minimum_required( VERSION 3.21 FATAL_ERROR )

list( APPEND CMAKE_MODULE_PATH "${CMAKE_SOURCE_DIR}/cmake" )
project( JoypadSpriteDemo
         LANGUAGES CA65816 )

add_executable( ${PROJECT_NAME}
    JoypadSprite.s )
set_target_properties( ${PROJECT_NAME}
    PROPERTIES SUFFIX ".smc" )
```

First, we add the `cmake` subdirectory to the paths where CMake searches for modules.

Open a command line, navigate to your project root directory, and run:

```
$ cmake -S . -B build
-- Building for: Ninja
-- Configuring done
-- Generating done
-- Build files have been written to: C:/Users/clc_x/code/retrodev/snesd

$ cmake --build build
[2/2] Linking CA65816 executable JoypadSpriteDemo.smc
```

Depending on your command line and OS, the output may slightly differ from what is shown here. If your build is successful, there should now be a file called `JoypadSpriteDemo.smc` in the subdirectory `build`. This is the exact same ROM we created in the [last tutorial][1].

We have successfully automated the build process for the ROM. This may seem a lot of work for compiling a single file into a ROM. But when we refactor the demo code by splitting it into several files, the advantage will become clear.

Fixing The Memory Map File Path

We'll now fix the hard-coded path to the memory map file that the linker needs (the `-C` argument). We'll simply use a CMake variable named `MEMORY_MAP_FILE` for this purpose. Change the two following files:

```
# snes_cradle/cmake/CMakeCA65816Information.cmake

# How to build objects
set( CMAKE_CA65816_COMPILE_OBJECT
    "<CMAKE_CA65816_COMPILER> --cpu 65816 \
                              -s \
                              -o <OBJECT> \
                              <SOURCE>"
)

# How to build executables
set( CMAKE_CA65816_LINK_EXECUTABLE
    "ld65 -C ${MEMORY_MAP_FILE} \
          <OBJECTS> \
```

```
            -o <TARGET>"
)

set( CMAKE_CA65816_INFORMATION_LOADED 1 )


# snes_cradle/CMakeLists.txt

cmake_minimum_required( VERSION 3.21 FATAL_ERROR )

list( APPEND CMAKE_MODULE_PATH "${CMAKE_SOURCE_DIR}/cmake" )
project( JoypadSpriteDemo
         LANGUAGES CA65816 )

add_executable( ${PROJECT_NAME}
    JoypadSprite.s )
set_target_properties( ${PROJECT_NAME}
    PROPERTIES SUFFIX ".smc" )
set( MEMORY_MAP_FILE ${CMAKE_CURRENT_SOURCE_DIR}/MemoryMap.cfg )

# Check if the path to the memory map file is set
if( NOT DEFINED MEMORY_MAP_FILE )
  message( FATAL_ERROR "Path to memory map file not set!" )
endif( NOT DEFINED MEMORY_MAP_FILE )
```

The Last four lines are just a simple sanity check to make sure you have the memory map file set (else, ld65 will give a weird error message that is hard to decipher; this makes it clear what the problem is).

Your directory structure now looks like this:

```
snes_cradle/
    cmake/
        CMakeCA65816Compiler.cmake.in
        CMakeCA65816Information.cmake
        CMakeDetermineCA65816Compiler.cmake
        CMakeTestCA65816Compiler.cmake
```

```
    CMakeLists.txt
    JoypadSprite.s
    MemoryMap.cfg
    SpriteColors.pal
    Sprites.vra
```

Run CMake, and load the result `build/JoypadSpriteDemo.smc` into an emulator to verify everything works. One last step is necessary before we can finally start refactoring `JoypadSprite.s`. We need to move all code into its own directory, as we plan to split it into multiple files.

First, create a new subdirectory in `snes_cradle` called `src` (or any other name you like, just keep it consistent). Move `JoypadSprite.s`, `SpriteColors.pal`, and `Sprites.vra` into this new subdirectory. In the new subdirectory `snes_cradle/src`, create a new file called `CMakeLists.txt`:

```
# snes_cradle/src/CMakeLists.txt

cmake_minimum_required( VERSION 3.21 FATAL_ERROR )

target_sources( ${PROJECT_NAME}
    PRIVATE JoypadSprite.s )
```

This file uses the `target_sources` command to tell CMake which files belong to which target. We only have one target in this project, which is the `JoypadSpriteDemo.smc` ROM. As we only have one source file (`JoypadSprite.s`, namely) so far, we add it here. Here's where we'll tell CMake which additional files to assemble after we split up `JoypadSprite.s`.

Finally, update `snes_cradle/CMakeLists.txt`:

```
# snes_cradle/CMakeLists.txt

cmake_minimum_required( VERSION 3.21 FATAL_ERROR )

list( APPEND CMAKE_MODULE_PATH "${CMAKE_SOURCE_DIR}/cmake" )

project( JoypadSpriteDemo
```

```
        LANGUAGES CA65816 )

add_executable( ${PROJECT_NAME} "" )
add_subdirectory( src )
set_target_properties( ${PROJECT_NAME}
    PROPERTIES SUFFIX ".smc" )
set( MEMORY_MAP_FILE ${CMAKE_CURRENT_SOURCE_DIR}/MemoryMap.cfg )

# Check if the path to the memory map file is set
if( NOT DEFINED MEMORY_MAP_FILE )
  message( FATAL_ERROR "Path to memory map file not set!" )
endif( NOT DEFINED MEMORY_MAP_FILE )
```

We now create a CMake target with no sources so far (`add_executable( ${PROJECT_NAME} "" )`). The added `add_subdirectory( src )` command tells CMake to look for a `CMakeLists.txt` in the given path and execute it. So in our case, CMake will look for `src/CMakeLists.txt` and execute it.

Your directory structure should look like this now:

```
snes_cradle/
    cmake/
        CMakeCA65816Compiler.cmake.in
        CMakeCA65816Information.cmake
        CMakeDetermineCA65816Compiler.cmake
        CMakeTestCA65816Compiler.cmake
    src/
        CMakeLists.txt
        JoypadSprite.s
        SpriteColors.pal
        Sprites.vra
    CMakeLists.txt
    MemoryMap.cfg
```

Rerun CMake (`cmake --build build`) to ensure everything works. Reload the resulting ROM file into an emulator. Everything works? Nice!

Conclusion

We now have a useful basic project structure we can use in future tutorials. Some may feel using CMake is overkill here (they'll prefer simple makefiles maybe). But I think it will simplify overall development as integrating new files and assets into the project will become a matter of adding the filenames to the correct command, and CMake will take care of the rest.

I kept the CMake parts purposefully simple. There's additional checks and features you could add, like different target configurations to generate ROM files that can be sent to a chip burner.

Next time, we'll further automate our build process by using CMake commands to automatically create asset and header files. And we'll finally refactor our sprite demo into smaller, reusable pieces.

Thank you for reading and see you next time!

Links and References

- Find the code of this tutorial on Github
- A good introduction to CMake
- A more in-depth introduction called Modern CMake
- Fun drinking game for when your build system is torturing you

ALSO ON MACHINE CODE CONSTRUCTION YARD

SNES Assembly Adventure 11: ...
4 years ago · 2 comments
I do assembly programming on old machines and boat building.

SNES Assembly Adventure 03: ...
8 years ago · 22 comments
I do assembly programming on old machines and boat building.

SNES Assembly Adventure 10: ...
4 years ago · 5 comments
I do assembly programming on old machines and boat building.

A CMake Ter
Game Boy D
3 years ago · 1 c
I do assembly
on old machin
building.